Benchmark Classification Models for Music Genre Prediction

Back

Build

Domain

Data Science

Machine Learning

Roles

Data Scientist

Skills

Benchmarking Classification Modeling Model Evaluation Model Serialization Public Relations Source Control

Difficulty

Easy

Tools

Git joblib Jupyter Matplotlib Numpy Pandas Python Scikit-Learn Seaborn

Pending Submission

Please submit your work when ready.

TIME REMAINING

1d

Deadline: **7 Nov 2026 at 04:59 PM**

Overview Instructions Resources Submit

Objective

You will build a machine learning pipeline to preprocess audio features, train multiple classification models, benchmark their performance, and select the best one for predicting music genres. You will learn feature scaling, training and evaluating multiple classifiers, model benchmarking techniques using metrics like F1-score, and model serialization for persistence. This process is fundamental

in applied machine learning for selecting the most effective algorithm for a specific problem, from content recommendation to medical diagnosis.

Description

1. Project Background and Objective

Welcome to the world of music information retrieval! In this project, you will tackle a classic machine learning problem: classifying music into genres based on its audio features. Streaming services like Spotify and Apple Music use sophisticated models to automatically categorize new tracks, enhance music discovery, and power personalized playlists. Your task is to build and compare several standard classification models to determine which one performs best for this task.

This project focuses on three critical skills for any data scientist: **Model Benchmarking**, **Model Serialization**, and **Source Control**.

Core Concepts

1. **Model Benchmarking**: This is the process of systematically evaluating and comparing the performance of multiple machine learning models to determine which one is the most suitable for a given problem. Simply training one model isn't enough; you need to prove it's the right choice. This involves training several different algorithms (e.g., Logistic Regression, Support Vector Machines, Random Forests) on the same dataset and comparing their performance using a consistent set of evaluation metrics (like accuracy, precision, recall, and F1-score). This empirical approach ensures you select a model based on evidence, not just intuition.

2. **Model Serialization**: After you've trained a model, it exists only in your computer's memory. If you close your program, the trained model is lost. Serialization (also known as saving or pickling) is the process of converting your trained model object into a file format that can be stored on disk. This allows you to save your work, share the model with others, or load it later into a different application (like a web API) to make predictions on new data without having to retrain it from scratch. Common libraries for this in Python are `pickle` and `joblib`.

3. **Source Control (Git)**: In any data science project, your code, notebooks, and results will change over time. Source control systems like Git allow you to track these changes, collaborate with others, and revert to previous versions if something goes wrong. Maintaining a clean, well-structured Git repository

is a professional standard. It documents your workflow and makes your work reproducible.

By completing this task, you will demonstrate your ability to manage a full, albeit small, machine learning project lifecycle from data preparation to model selection and persistence.

2. The Dataset: GTZAN Music Genre Features

To keep the focus on modeling rather than complex audio processing, you will use a pre-processed version of the famous GTZAN Genre Collection dataset. This dataset consists of a CSV file where each row represents a 30-second music clip, and the columns contain various extracted audio features.

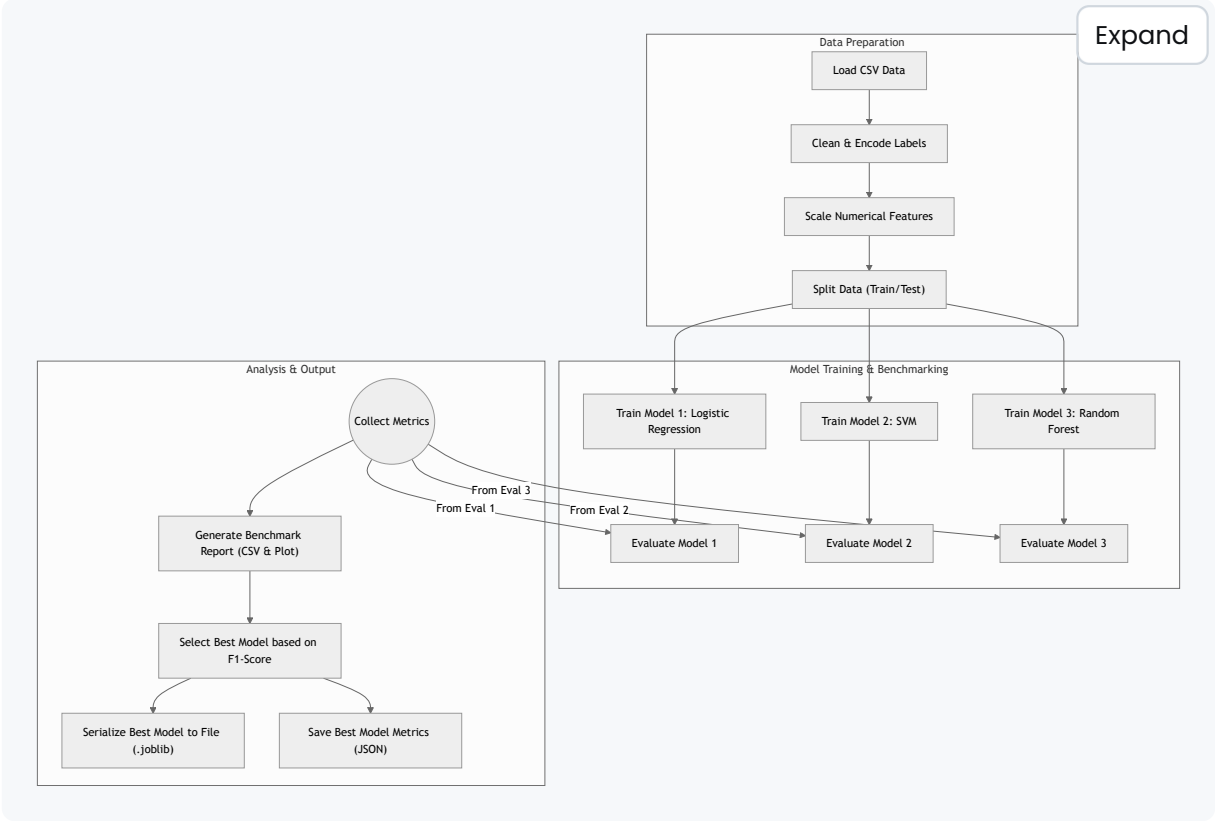
You will be provided with a link to download `features_30_sec.csv`. The key columns include:

1. **Technical Audio Features**: `chroma_stft_mean`, `rmse_mean`, `spectral_centroid_mean`, `spectral_bandwidth_mean`, `rolloff_mean`, `zero_crossing_rate_mean`, and many others, including Mel-Frequency Cepstral Coefficients (MFCCs).
2. **Target Label**: `label`, a string representing the music genre (e.g., 'blues', 'classical', 'jazz').

Your first step will be to explore this data to understand the feature distributions and their relationship with the genre labels.

3. High-Level Project Workflow

Your project will follow a structured workflow, which is a standard practice in applied machine learning projects. The diagram below illustrates the key stages from data ingestion to the final selection and serialization of your best model.



Following this workflow will help you organize your code logically and ensure all requirements are met systematically.

4. Step-by-Step Implementation Guide

Here is a recommended path to complete the project. You can organize your work in Jupyter Notebooks for exploration and Python scripts for reproducible steps like preprocessing and training.

Step 1: Project Setup and Environment

1. **Initialize a Git Repository**: Create a new directory for your project and initialize a Git repository. Create an initial commit.
2. **Create Project Structure**: Set up the required directory structure as specified in the core requirements (`data`, `output`, `src`, `notebooks`).
3. **Set up Python Environment**: Create a virtual environment (e.g., using `venv` or `conda`) to manage your project's dependencies. This isolates your project from your system's Python installation.
4. **Install Libraries**: Install the necessary Python libraries, such as `pandas`, `numpy`, `scikit-learn`, `matplotlib`, and `seaborn`. Remember to add these to a `requirements.txt` file.
5. **Gitignore**: Create a `.gitignore` file to prevent tracking unnecessary files (e.g., virtual environment folders, `__pycache__` directories, large data files).

Step 2: Data Exploration and Preprocessing

1. **Load Data**: Place the dataset CSV in your `data/` directory and load it into a pandas DataFrame.
2. **Exploratory Data Analysis (EDA)**: In a Jupyter Notebook (`notebooks/eda.ipynb`), explore the dataset. Check for missing values, examine the distribution of genres (the target variable), and visualize the distributions of a few key features. This will give you an intuition for the data you're working with.
3. **Label Encoding**: The `label` column is categorical. Machine learning models require numerical input, so you will need to convert the text labels into integers using `sklearn.preprocessing.LabelEncoder`.
4. **Feature Scaling**: The numerical features in the dataset are on different scales. This can negatively impact the performance of some models, like SVMs. Use `sklearn.preprocessing.StandardScaler` to scale all feature columns to have zero mean and unit variance. Apply the scaler by fitting it **only on the training data** and then transforming both the training and testing data to prevent data leakage.
5. **Train-Test Split**: Split your dataset into a training set and a testing set using `sklearn.model_selection.train_test_split`. A common split is 80% for training and 20% for testing. Use a fixed `random_state` to ensure your split is reproducible.

Step 3: Model Training and Benchmarking

1. **Define Models to Benchmark**: Choose at least three different classification algorithms from scikit-learn. We recommend starting with:
    1. `LogisticRegression`
    2. `SVC` (Support Vector Classifier)
    3. `RandomForestClassifier`
2. **Train Models**: Train each of your chosen models on the preprocessed *training* data.
3. **Make Predictions**: For each trained model, generate predictions on the *testing* data.
4. **Calculate Evaluation Metrics**: For each model, calculate the following metrics using the test set predictions and true labels:
    1. **Accuracy**: Overall correct predictions.
    2. **Precision, Recall, and F1-Score**: For multiclass problems, it's important to look at these metrics. Use the **macro-averaged** versions to ensure you are evaluating performance across all classes equally, without bias towards the most populous ones. You can find these in `sklearn.metrics.classification_report` or calculate them individually.

Step 4: Results Analysis and Model Serialization

1. **Consolidate Results**: Create a pandas DataFrame to store the performance metrics for all models. This DataFrame will be saved as your `benchmark_results.csv` file.
2. **Visualize Comparison**: Create a bar chart using `matplotlib` or `seaborn` to visually compare a key metric (e.g., F1-score) across the different models. Save this plot as an image file.
3. **Select the Best Model**: Based on your benchmark results, identify the best-performing model. For this task, the primary decision metric should be the **macro-averaged F1-score**, as it provides a balanced measure of precision and recall.
4. **Serialize the Model**: Once you've identified the best model, use `joblib.dump` (recommended for scikit-learn models) to serialize the trained model object and save it to a file named `best_model.joblib` in your `output/` directory.
5. **Save Final Metrics**: Save the detailed metrics of your chosen best model into a separate JSON file, `output/best_model_metrics.json`.

5. Documentation and Submission

Finally, ensure your project is well-documented and ready for submission.

1. **README File**: Your `README.md` is the front page of your project. It should clearly explain what the project is about, how to set it up (clone repo, create environment, install dependencies), and how to run your scripts to reproduce the results. Include a summary of your benchmark findings.
2. **Final Code**: Clean up your notebooks and scripts. Ensure they are well-commented and follow a logical flow.
3. **Commit and Push**: Make sure all your final changes are committed to Git and pushed to your remote repository (e.g., on GitHub).

6. FAQ

1. **Q: Why do I need to scale my features?**
    1. A: Many algorithms, especially those that use distance calculations like SVM or regularization like Logistic Regression, are sensitive to the scale of input features. Features with larger ranges can dominate the learning process. Scaling brings all features to a comparable range, ensuring they contribute more equally. For more detail, read the scikit-learn documentation on preprocessing.

2. **Q: What is 'data leakage' and why is fitting the scaler on the full dataset bad?**

    1. A: Data leakage is when information from outside the training dataset is used to create the model. If you fit the `StandardScaler` on the entire dataset (including the test set), the scaler learns the mean and standard deviation from the test data. This gives your model a sneak peek at the test set's properties, leading to overly optimistic performance estimates. You must only fit on training data and use that fitted scaler to transform both train and test sets. Read more on this common pitfall here.

3. **Q: Why use macro-averaging for precision/recall/F1?**

    1. A: In a multiclass classification problem, some classes might have many more samples than others. A simple average (`micro-average`) would be dominated by the model's performance on the larger classes. A `macro-average` calculates the metric independently for each class and then takes the average, treating all classes equally. This gives a better sense of how the model performs across all genres, not just the most common ones. See the scikit-learn metrics documentation for a detailed explanation.

4. **Q: `pickle` vs. `joblib`? Which should I use?**

    1. A: `joblib` is generally more efficient and robust for saving objects that contain large NumPy arrays, which is common for scikit-learn models. The scikit-learn documentation itself recommends using `joblib` over `pickle` for their objects. You can read their guide on Model Persistence.

Core Requirements

1. Your project repository must follow a standardized directory structure to ensure organization and reproducibility.

   Required Directory Structure:

   ```
   /
   ├── data/         # For raw and processed data
   ├── notebooks/    # For Jupyter notebooks for exploration (EDA)
   ├── output/       # For all generated files (models, plots, reports)
   ├── src/          # For Python source code scripts
   ├── .gitignore
   ├── README.md
   └── requirements.txt
   ```

   Verification:

   1. The repository must contain the following directories at the root level: `data`, `notebooks`, `output`, `src`.
   2. The repository root must contain the files: `.gitignore`, `README.md`, and `requirements.txt`.

---

2. A `requirements.txt` file must be present to document all Python package dependencies for the project.

   File Location: `requirements.txt` (in the project root)

   Content: This file should list all external Python libraries required to run your code, including but not limited to:

   1. `pandas`
   2. `numpy`
   3. `scikit-learn`
   4. `matplotlib`
   5. `seaborn`
   6. `joblib`

   Each dependency should be on a new line, preferably with a version number (e.g., `pandas==2.0.3`).

   Verification:

   1. The file `requirements.txt` must exist in the project root.
   2. The file must contain entries for `scikit-learn` and `pandas`.

---

3. A `.gitignore` file must be configured to exclude unnecessary files and directories from source control.

   File Location: `.gitignore` (in the project root)

   Required Entries: The file must include rules to ignore:

   1. Python virtual environment directories (e.g., `venv/`, `.venv/`)
   2. Python bytecode files (`__pycache__/`, `*.pyc`)
   3. The dataset file itself to avoid committing large files to Git (`data/*.csv`)

   Verification:

   1. The file `.gitignore` must exist in the project root.
   2. The file's content will be checked to ensure it contains patterns for `__pycache__`, a common virtual environment folder name like `venv`, and `data/*.csv`.

---

4. You must perform and document your exploratory data analysis (EDA) in a Jupyter Notebook.

   File Location: `notebooks/eda.ipynb`

   Content: The notebook should contain:

   1. Code to load the dataset.
   2. Analysis of the target variable distribution (genres).
   3. Basic statistics and checks for missing values.
   4. At least one visualization of a feature's distribution (e.g., a histogram).

   Verification:

   1. The file `notebooks/eda.ipynb` must exist.
   2. The file must be a valid Jupyter Notebook that can be opened and reviewed.

---

5. The core logic for data preprocessing and model training must be encapsulated in reproducible Python scripts.

   File Location: `src/` directory

   Required Scripts:

   1. A script responsible for data preprocessing (e.g., `src/preprocess.py`).
   2. A script responsible for training the models and generating outputs (e.g., `src/train.py`).

   Verification:

   1. At least one `.py` file must exist inside the `src/` directory.
   2. This requirement checks for the presence of organized, script-based logic, separate from the exploratory notebook.

---

6. Your analysis must produce a CSV file summarizing the performance of all trained models. This file is the primary artifact for model comparison.

   File Location: `output/benchmark_results.csv`

   Required Columns: The CSV file must contain the following columns, though the order does not matter:

Column	Data Type	Description
`model_name`	string	The name of the model (e.g., 'Logistic Regression', 'SVM').
`accuracy`	float	The accuracy score on the test set.
`precision`	float	The macro-averaged precision score on the test set.
`recall`	float	The macro-averaged recall score on the test set.
`f1_score`	float	The macro-averaged F1-score on the test set.

   Verification:

   1. The file `output/benchmark_results.csv` must exist.
   2. The file must be a valid CSV.
   3. The CSV header must contain the exact column names: `model_name`, `accuracy`, `precision`, `recall`, `f1_score`.
   4. The file must contain at least three rows of data, one for each benchmarked model.
   5. The data types for the metric columns must be floating-point numbers.

---

7. A plot visualizing the model comparison must be saved as a static image file.

   File Location: `output/benchmark_comparison.png`

   Content: The plot should be a bar chart (or similar) that clearly compares the performance of the trained models using one of the key metrics (preferably the F1-score).

   Verification:

   1. The file `output/benchmark_comparison.png` must exist.
   2. The file must be a valid PNG image.

---

8. You must save the evaluation metrics for the single best-performing model to a structured JSON file.

   File Location: `output/best_model_metrics.json`

   Required JSON Structure:

```
{
  "model_name": "string",
  "accuracy": "float",
  "precision": "float",
  "recall": "float",
  "f1_score": "float"
}
```

   Verification:

   1. The file `output/best_model_metrics.json` must exist.
   2. The file must be valid JSON and parse without errors.
   3. The JSON object must contain the exact keys: `model_name`, `accuracy`, `precision`, `recall`, and `f1_score`.
   4. The value for `model_name` must be a string, and the values for the metrics must be floating-point numbers.

---

9. The best-performing trained model object must be serialized to a file for persistence.

   File Location: `output/best_model.joblib`

   Content: This file should be the result of serializing your final, trained scikit-learn model object using a library like `joblib`.

   Verification:

   1. The file `output/best_model.joblib` must exist.
   2. The presence of this file demonstrates successful model serialization.

---

10. Your project must include a high-quality `README.md` file that documents the project.

    File Location: `README.md` (in the project root)

    Required Sections: The README must contain the following sections, identified by markdown headers:

    1. A section for project overview/description.
    2. A section with setup and installation instructions (e.g., "Setup", "Installation").
    3. A section summarizing the results (e.g., "Results", "Findings").

    Verification:

    1. The file `README.md` must exist.
    2. The file will be parsed to check for the presence of headers indicating the required sections.

All rights reserved. Copyright, Partnr 2025-26

